

Acumuladores Booleanos

Programación II

Definición

- Una variable booleana, conceptualmente, es igual a una variable entera o real, con las que estamos muy familiarizados.
- La diferencia obvia es que mientras una variable entera puede tomar infinitos valores diferentes, una variable booleana sólo puede asumir dos posibles valores o valores de verdad.
- Un acumulador es una variable que se usa durante la vida de un algoritmo, generalmente dentro de los ciclos, para acumular resultados que se producen dentro de un bucle o, varían en un valor constante en cada uno.
- Un acumulador booleano utiliza las propiedades de la lógica para acumular un valor de verdad (verdadero o falso) durante la vida de un algoritmo. En general estos algoritmos son una implementación de una función booleana.

Tablas de Verdad

- Sean a y b dos variables booleanas. Las propiedades lógicas para las operaciones AND(&&) y OR(||) son:

	a	b
(1)	f	f
(2)	f	v
(3)	v	f
(4)	v	v

Tablas de Verdad

- Sean a y b dos variables booleanas. Las propiedades lógicas para las operaciones AND(&&) y OR(||) son:

	a	b	A b	a && b
(1)	f	f	f	f
(2)	f	v	v	f
(3)	v	f	v	f
(4)	v	v	v	v

Ejemplo 1

- Queremos ver si toda una lista de números es par. Por ejemplo:

```
- Arreglo A: [4 7 9 15 35 39] retorna false
- Arreglo B: [4 8 10 2 12] retorna true
```

Cuantificadores

Cuando queremos probar una propiedad P para todo un conjunto de datos:

$$\{\forall x \in lista / P(x)\} \equiv true$$

Utilizaremos la hipótesis de $ret = \textbf{true}$ y la acumulación será de la forma:

$$ret = ret \text{ AND } P(x)$$

Cuando queremos probar una propiedad P para un solo elemento:

$$\{\exists x \in lista / P(x)\} \equiv true$$

Utilizaremos la hipótesis de $ret = \textbf{false}$ y la acumulación será de la forma:

$$ret = ret \text{ OR } P(x)$$

Ejemplo 1

- Queremos ver si toda una lista de números es par. Por ejemplo:

```
- Arreglo A: [4 7 9 15 35 39] retorna false
- Arreglo B: [4 8 10 2 12] retorna true
```

Ejemplo 1 - Versión 1a, Sin utilizar acumuladores

```
boolean esPar(lista)
for (int i=0, i++, i < lista.size())
    if !(par(lista[i]))
        return false
return true
```

Ejemplo 1 - Versión 1b, Utilizando acumuladores:

```
boolean esPar(lista)
boolean ret = true
for (int i=0, i++, i < lista.size())
    ret = ret && par(lista[i])
return ret
```

Ejemplo 2 - Versión 2a, Sin utilizar acumuladores

- Ahora, además de decir si la lista es par o no lo es, también queremos dejar siempre la lista con todos sus elementos multiplicados por 2

```
boolean esPar(lista)
for (int i=0, i++, i < lista.size())
    if !(par(lista[i]))
        return false
    lista[i] = lista[i] * 2
return true
```

Ejemplo 2 - Versión 2a, Sin utilizar acumuladores

- Ahora, además de decir si la lista es par o no lo es, también queremos dejar siempre la lista con todos sus elementos multiplicados por 2

```
boolean esPar(lista)
for (int i=0, i++, i < lista.size())
    if !(par(lista[i]))
        return false
    lista[i] = lista[i] * 2
return true
```

Pero esto no siempre funciona. Si no llegamos hasta el final de la lista, no multiplicaremos el resto de la lista por 2. V

Ejemplo 2 - Versión 2b, sin acumuladores:

- Ahora, además de decir si la lista es par o no lo es, también queremos dejar siempre la lista con todos sus elementos multiplicados por 2

```
boolean esPar(lista)
for (int i=0, i++, i < lista.size())
    lista[i] = lista[i] * 2
for (int i=0, i++, i < lista.size())
    if !(par(lista[i]))
        return false
return true
```

Pero esto no siempre funciona. ¡Al multiplicar por 2, todas las listas quedan pares! Pero no es cierto que la lista original sea par. Ej. [1,3,5] es impar, mientras que [2,6,10] es par! y luego de multiplicarla por dos devuelve true, lo que es un error.

Ejemplo 2 - Versión 2c, sin acumuladores

- Ahora, además de decir si la lista es par o no lo es, también queremos dejar siempre la lista con todos sus elementos multiplicados por 2

```
boolean esPar(lista)
    copiaLista = lista.clonar()
    for (int i=0, i++, i < lista.size())
        lista[i] = lista[i] * 2
    for (int i=0, i++, i < copiaLista.size())
        if !(par(copiaLista[i]))
            return false
    return true
```

Finalmente funcionó, ¿pero a qué costo?

Ejemplo 2 - Versión 2d, con acumuladores

- Ahora, además de decir si la lista es par o no lo es, también queremos dejar siempre la lista con todos sus elementos multiplicados por 2

```
boolean esPar(lista){  
    boolean ret = true;    ← Notar que el acumulador inicia en true  
    for (int i=0, i++, i < lista.size()){  
        ret = ret && par(lista[i]);  
        lista[i] = lista[i] * 2;}  
    return ret;} 
```

Ejemplo 3

- Queremos ver si algún número de una lista es par.

```
Arreglo A: [3 8 1 9 11] retorna true  
Arreglo B: [9 5 1 7 3] retorna false
```

En este ejemplo se utilizará el operador lógico OR.

Ejemplo 3 - Versión 3a, sin acumuladores

- Queremos ver si algún número de una lista es par.

```
boolean algunPar(lista) {  
    for (int i=0, i++, i < lista.size())  
        if (par(lista[i]))  
            return true;  
    return false;  
}
```

Ejemplo 3 - Versión 3b, con acumuladores

- Queremos ver si algún número de una lista es par.

```
boolean algunPar(lista) {  
    boolean ret = false; ← Notar que el acumulador inicia en false  
    for (int i=0, i++, i < lista.size())  
        ret = ret || par(lista[i]);  
    return ret;  
}
```

Regla general para acumulación booleana

1. Cuando la hipótesis es: que la propiedad se cumple para todos los elementos, ret comienza valiendo true y la acumulación es con “and”.
2. Cuando la hipótesis es: que la propiedad se cumple para al menos un elemento, ret comienza valiendo false y la acumulación es con “or”.

Ejercicio

Realizar una función que dada una lista de números devuelva verdadero si:

- La lista tiene todos los números mayores que 8
- La lista tiene algún número menor que 23
- Utilizando acumuladores boléanos

Ejercicio - Solucion

```
public class Ppal {

    public static void main(String[] args) {
        Integer [] lista1 = {9,10,11};           //cumple
        Integer [] lista2 = {90,100,110};        //no cumple

        System.out.println(todosMayor8(lista1) && algunoMenor23(lista1)); //true
        System.out.println(todosMayor8(lista2) && algunoMenor23(lista2)); //false
    }

    static public boolean todosMayor8(Integer [] lista){
        boolean ret = true;
        for (int i = 0; i < lista.length; i++){
            ret = ret && lista[i] > 8;
        }
        return ret;
    }

    static public boolean algunoMenor23(Integer [] lista){
        boolean ret = false;
        for (int i = 0; i < lista.length; i++){
            ret = ret || lista[i] < 23;
        }
        return ret;
    }
}
```

Conclusión Importante:

Cuando una propiedad se quiere probar sobre toda la lista, se utiliza “AND”

Cuando una propiedad se quiere demostrar sobre algún elemento de la lista, se utiliza “OR”